# Adaptability questions

*Design Gurus*

Here are a few behavioral questions related to problem solving and decision making:

## Question 1: How do you approach problem-solving and debugging when faced with a complex issue in your code?

**Intention of the question:** The interviewer wants your actual method, not a description of the scientific method. They are listening for how you narrow a problem down, and how you find out that a theory is wrong.

**Pitfalls to avoid:** Avoid describing a general process with no example attached to it. Do not pick a bug you solved by guessing correctly. Say what you ruled out and how you ruled it out, because that is where the skill is.

**Sample Answer 1:**

**Situation:** "We had a service that returned the wrong balance for about one request in fifty thousand. It had been open for five weeks and two engineers had already looked at it. Nothing appeared in the logs, because from the code's point of view nothing had failed."

**Task:** "I was asked to take it because customer complaints had reached about twelve a week. The hard part was reproduction. I could not make it happen on demand, so I could not test a fix even if I found one."

**Action:** "I stopped trying to reproduce it and started trying to catch it. I added a check that recomputed the balance a second time before responding. It logged both values whenever they disagreed. That cost about 3 milliseconds per request, and I got approval to run it for 48 hours. It caught nine cases. All nine had two writes to the same account inside one 20 millisecond window. That ruled out our three theories at once. Those were a stale cache, a rounding error, and a bad database replica. Once I had the pattern, reproducing it in a test took about an hour."

**Result:** "The cause was a read served by a replica while the write went to the primary. I fixed it by reading through the primary for balance requests only. That added about 8 milliseconds to one endpoint. Wrong balances went to zero over the next three months and the complaints stopped. When a bug is rare, instrument first and theorize second. Five weeks of theories produced nothing and 48 hours of logging produced the answer."

**Sample Answer 2:** "A different bug taught me the opposite lesson. Our nightly job started failing about once a week, always with a timeout. I assumed it was load and spent two days on query performance, which changed nothing. Then I plotted the failures against the calendar, and every one of them was a Monday. Monday's run covered the weekend, so it processed roughly three times the rows. The fix was a batch size, not a query. I had lost two days because I asked what was slow about the job instead of asking what the failing runs had in common. I now start with that second question."

**Follow-up you should expect:** "You added logging to production to catch it. How did you decide that was safe?"

**Why they ask:** They want to know whether you weigh the cost of your own diagnostic tools, or whether you reach into production by default.

**How to answer it:** "I measured it first. The extra computation was about 3 milliseconds against a p99 of 180, so under 2 percent. I put it behind a flag so I could switch it off in one step. I ran it on 10 percent of traffic for an hour before I ran the full 48 hours. It also logged only when the two values disagreed, so a normal request wrote nothing. If the cost had been 30 milliseconds instead of 3, I would have sampled instead, and accepted a week to collect those nine cases."

## Question 2: How do you handle situations where you are unsure of how to proceed or solve a problem? Can you provide an example?

**Intention of the question:** The interviewer wants to know how long you stay stuck before you ask for help. They also want to know what you do with the time in between. They are checking for judgment about when help is cheaper than persistence.

**Pitfalls to avoid:** Avoid suggesting you always work it out alone, which reads as expensive rather than capable. Avoid suggesting you ask immediately, which reads as unable to start. Do not describe asking for help without saying what you had already tried.

**Sample Answer 1:**

**Situation:** "I was asked to cut the cost of our search cluster, which was running at about 18,000 dollars a month. I had never operated that search engine and nobody at the company had tuned it. There was no obvious place to start."

**Task:** "The target was a 30 percent reduction within the quarter. I had no idea whether that was easy or impossible. That was the real problem. I could not plan the work until I knew which of those two it was."

**Action:** "I gave myself two days to find out, rather than an open-ended investigation. In those two days I did not try to fix anything. I measured where the money went. Seventy percent of it was one index holding six years of logs. Queries older than 90 days were about 2 percent of traffic. Then I stopped and asked for help. The next question was whether we were legally required to keep six years. That was not something I could work out from the code. Our compliance lead answered it in ten minutes: two years, not six."

**Result:** "Moving everything older than 90 days to cold storage cut the bill to about 11,000 dollars a month. That is a 39 percent reduction, and it took three weeks rather than a quarter. The two-day limit is the part I kept. I was not stuck because the problem was hard. I was stuck because I did not know which kind of problem it was. That is answerable in two days, not two weeks."

**Sample Answer 2:** "On a smaller scale, I have a rule for being stuck on a bug. If 45 minutes pass with no new information, I ask. New information counts, so a failed theory resets the clock. Before I ask, I write down what I have ruled out and what I would try next. About half the time writing that summary shows me the answer and I never send it. When I do send it, the colleague usually replies in a couple of minutes. I have done the narrowing for them. That rule has saved

me the two-day silences I used to have early in my career."

**Follow-up you should expect:** "Was there a time you asked for help too late?"

**Why they ask:** The rule you just described sounds tidy. They want to know whether it came from experience or from a book.

**How to answer it:** "Yes, and it is where the rule came from. I spent nine days on a build failure that nobody had asked me to fix. I had decided it was mine to solve. When I finally described it in our team channel, a colleague recognized it in about four minutes. It was a known problem with a version of a tool he had hit the year before. Nine days against four minutes is the number I remember. What stopped me asking was not wanting to look like I could not handle it. That cost far more than looking that way would have."